

A estrutura árvore de busca suporta muitas operações de conjuntos dinâmicos, incluindo SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT e DELETE. Assim, uma árvore de busca pode ser usada como um dicionário e também como uma fila de prioridades.

As operações básicas em uma árvore de busca binária demoram um tempo proporcional à altura da árvore. No caso de uma árvore binária completa com n nós, tais operações são executadas no tempo $O(\lg n)$ do pior caso. Porém, se a árvore é uma cadeia linear de n nós, as mesmas operações demoram o tempo $O(n)$ do pior caso. Veremos na Seção 12.4 que a altura esperada de uma árvore de busca binária construída aleatoriamente é $O(\lg n)$, de modo que as operações básicas de conjuntos dinâmicos em tal árvore demoram o tempo $O(\lg n)$ em média.

Na prática, nem sempre podemos garantir que as árvores de busca binária sejam construídas aleatoriamente, mas podemos projetar variações de árvores de busca binária com bom desempenho do pior caso garantido em operações básicas. O Capítulo 13 apresenta uma dessas variações, as árvores vermelhopreto, que têm altura $O(\lg n)$. O Capítulo 18 introduz as árvores B (Btrees), que são particularmente boas para manter bancos de dados em armazenamento secundário (disco).

Depois da apresentação das propriedades básicas de árvores de busca binária, as próximas seções mostram como percorrer uma árvore de busca binária para imprimir seus valores em sequência ordenada, como procurar um valor em uma árvore de busca binária, como encontrar o elemento mínimo ou máximo, como encontrar o predecessor ou o sucessor de um elemento e como inserir ou eliminar elementos em uma árvore de busca binária. As propriedades matemáticas básicas das árvores são apresentadas no Apêndice B.

12.1 O QUE É UMA ÁRVORE DE BUSCA BINÁRIA?

Uma árvore de busca binária é organizada, como o nome sugere, em uma árvore binária, como mostra a Figura 12.1. Podemos representar tal árvore por uma estrutura de dados ligada, na qual cada nó é um objeto. Além de uma *chave* e de dados satélites, cada nó contém atributos *esquerda*, *direita* e *p*, que apontam para os nós correspondentes ao seu filho à esquerda, ao seu filho à direita e ao seu pai, respectivamente. Se um filho ou o pai estiver ausente, o atributo adequado contém o valor NIL. O nó raiz é o único nó na árvore cujo pai é NIL.

As chaves em uma árvore de busca binária são sempre armazenadas de modo a satisfazer a propriedade de árvore de busca binária:

Seja x um nó em uma árvore de busca binária. Se y é um nó na subárvore esquerda de x , então $y.chave \leq x.chave$. Se y é um nó na subárvore direita de x , então $x.chave \geq y.chave$.

Assim, na Figura 12.1(a), a chave da raiz é 6, as chaves 2, 5 e 5 em sua subárvore esquerda não são maiores que 6, e as chaves 7 e 8 em sua subárvore direita não são menores que 6. A mesma propriedade é válida para todo nó na árvore. Por exemplo, a chave 5 no filho à esquerda da raiz não é menor que a chave 2 na subárvore esquerda daquele nó e não é maior que a chave 5 na subárvore direita.

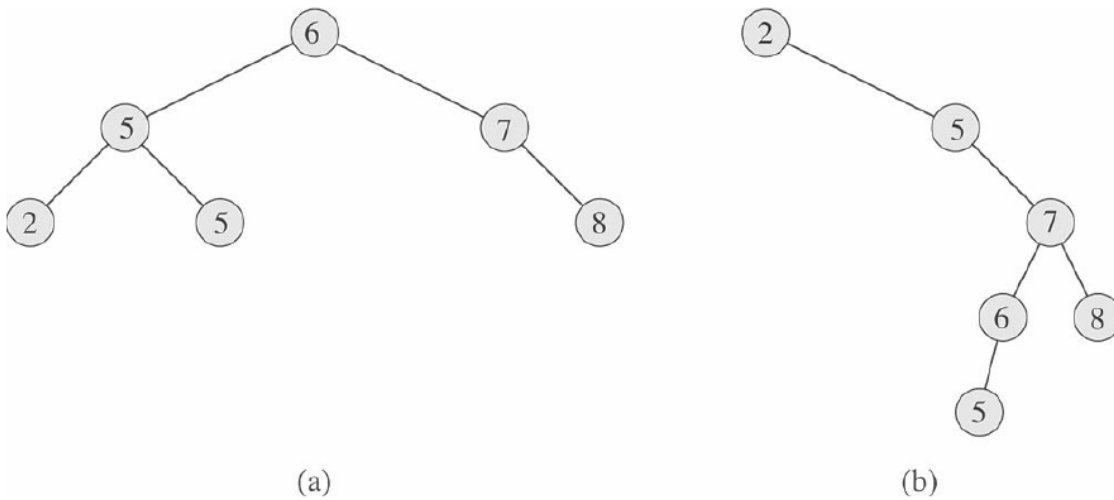


Figura 12.1 Árvores de busca binária. Para qualquer nó x , as chaves na subárvore esquerda de x são no máximo $x.chave$, e as chaves na subárvore direita de x são no mínimo $x.chave$. Árvores de busca binária diferentes podem representar o mesmo conjunto de valores. O tempo de execução do pior caso para a maioria das operações em árvores de busca é proporcional à altura da árvore. (a) Uma árvore de busca binária com seis nós e altura 2. (b) Uma árvore de busca binária menos eficiente, com altura 4, que contém as mesmas chaves.

A propriedade de árvore de busca binária nos permite imprimir todas as chaves em uma árvore de busca binária em sequência ordenada por meio de um simples algoritmo recursivo, denominado *percurso de árvore em in-ordem*. Esse algoritmo tem tal nome porque imprime a chave da raiz de uma subárvore entre a impressão dos valores em sua subárvore esquerda e a impressão dos valores em sua subárvore direita. (De modo semelhante, um *percurso de árvore em pré-ordem* imprime a raiz antes dos valores das subárvores, e um *percurso de árvore em pós-ordem* imprime a raiz depois dos valores em suas subárvores.) Para usar o procedimento a seguir com o objetivo de imprimir todos os elementos em uma árvore de busca binária T , chamamos `INORDER-TREE-WALK( $T.raiz$ )`.

```

INORDER-TREE-WALK( $x$ )
1  if  $x \neq \text{NIL}$ 
2    INORDER-TREE-WALK( $x.esquerda$ )
3    print  $x.chave$ 
4    INORDER-TREE-WALK( $x.direita$ )

```

Como exemplo, o percurso de árvore em ordem imprime as chaves em cada uma das duas árvores de busca binária da Figura 12.1 na ordem 2, 3, 5, 6, 7, 8. A correção do algoritmo decorre por indução diretamente da propriedade de árvore de busca binária.

Percorrer uma árvore de busca binária de n nós demora o tempo $Q(n)$ já que, após a chamada inicial, o procedimento chama a si mesmo recursivamente exatas duas vezes para cada nó na árvore — uma vez para seu filho à esquerda e uma vez para seu filho à direita. O teorema a seguir, apresenta uma prova formal de que o tempo para executar um percurso de árvore em inordem é linear.

Teorema 12.1

Se x é a raiz de uma subárvore de n nós, então a chamada `INORDER-TREE-WALK( $x$ )` demora o tempo $Q(n)$.

Prova Seja $T(n)$ o tempo tomado por `INORDER-TREE-WALK` quando chamado na raiz de uma subárvore de n nós. Visto que `INORDER-TREE-WALK` visita todos os n nós da subárvore, temos $T(n) = \Omega(n)$. Resta mostrar que $T(n) = O(n)$.

Uma vez que `INORDER-TREE-WALK` demora um tempo pequeno e constante em uma subárvore vazia (para o teste $x \neq \text{NIL}$), temos $T(0) = c$ para alguma constante $c > 0$.

Para $n > 0$, suponha que `INORDER-TREE-WALK` seja chamado em um nó x cuja subárvore esquerda tem k nós e cuja subárvore direita tem $n - k - 1$ nós. O tempo para executar `INORDER-TREE-WALK( $x$ )` é limitado por $T(n) = T(k) + T(n - k - 1) + d$ para alguma constante $d > 0$, que reflete um limite superior para o tempo de execução do corpo de `INORDER-TREE-WALK( $x$ )`, excluindo o tempo gasto em chamadas recursivas.

Usamos o método de substituição para mostrar que $T(n) = Q(n)$, provando que $T(n) \leq (c + d)n + c$. Para $n = 0$, temos $(c + d) \cdot 0 + c = c = T(0)$. Para $n > 0$, temos

$$\begin{aligned} T(n) &\leq T(k) + T(n - k - 1) + d \\ &= ((c + d)k + c) + ((c + d)(n - k - 1) + c) + d \\ &= (c + d)n + c - (c + d) + c + d \\ &= (c + d)n + c, \end{aligned}$$

o que conclui a prova.

Exercícios

- 12.1-1** Trace árvores de busca binária de alturas 2, 3, 4, 5 e 6 para o conjunto de chaves $\{1, 4, 5, 10, 16, 17, 21\}$.
- 12.1-2** Qual é a diferença entre a propriedade de árvore de busca binária e a propriedade de heap de mínimo (veja p. 153)? A propriedade de heap de mínimo pode ser usada para imprimir as chaves de uma árvore de n nós em sequência ordenada no tempo $O(n)$? Justifique sua resposta.
- 12.1-3** Dê um algoritmo não recursivo que execute um percurso de árvore em ordem. (*Sugestão:* Uma solução fácil usa uma pilha como uma estrutura de dados auxiliar. Uma solução mais complicada, porém elegante, não emprega nenhuma pilha, mas considera que é possível testar a igualdade entre dois ponteiros.)
- 12.1-4** Dê algoritmos recursivos que executem percursos de árvores em pré-ordem e pós-ordem no tempo $Q(n)$ em uma árvore de n nós.
- 12.1-5** Mostre que, considerando que a ordenação de n elementos demora o tempo $(n \lg n)$ no pior caso do modelo de comparação, qualquer algoritmo baseado em comparação para construir uma árvore de busca binária com base em uma lista arbitrária de n elementos demora o tempo $(n \lg n)$ no pior caso.

12.2 CONSULTAS EM UMA ÁRVORE DE BUSCA BINÁRIA

Frequentemente precisamos procurar uma chave armazenada em uma árvore de busca binária. Além da operação `SEARCH`, árvores de busca binária podem suportar as consultas `MINIMUM`, `INORDER-MAXIMUM`, `SUCCESSOR` e `PREDECESSOR`. Nesta seção, examinaremos essas operações e mostraremos como suportar cada uma delas no tempo $O(h)$ em qualquer árvore de busca binária de altura h .

Buscas

Usamos o procedimento a seguir para procurar um nó com determinada chave em uma árvore de busca binária. Dado um ponteiro para a raiz da árvore e uma chave k , `TREE-SEARCH` retorna um ponteiro para um nó com chave k , se existir algum; caso contrário, ele retorna `NIL`.

TREE-SEARCH(x, k)

```
1  if  $x == \text{NIL}$  ou  $k == x.chave$ 
2      return  $x$ 
3  if  $k < x.chave$ 
4      return TREE-SEARCH( $x.esquerda, k$ )
5  else return TREE-SEARCH( $x.direita, k$ )
```

O procedimento começa sua busca na raiz e traça um caminho simples descendo a árvore, como mostra a Figura 12.2. Para cada nó x que encontra, ele compara a chave k com a $x.chave$. Se as duas chaves são iguais, a busca termina. Se k é menor que $x.chave$, a busca continua na subárvore esquerda de x , já que a propriedade de árvore de busca binária implica que k não poderia estar armazenada na subárvore direita. Simetricamente, se k é maior que $x.chave$, a busca continua na subárvore direita. Os nós encontrados durante a recursão formam um caminho simples descendente partindo da raiz da árvore e, portanto, o tempo de execução de TREE-SEARCH é $O(h)$, onde h é a altura da árvore.

O mesmo procedimento pode ser reescrito de modo iterativo, “desdobrando” a recursão dentro de um laço **while**. Na maioria dos computadores, a versão iterativa é mais eficiente.

ITERATIVE-TREE-SEARCH(x, k)

```
1  while  $x \neq \text{NIL}$  e  $k \neq x.chave$ 
2      if  $k < x.chave$ 
3           $x = x.esquerda$ 
4      else  $x = x.direita$ 
5  return  $x$ 
```

Mínimo e máximo

Sempre podemos encontrar um elemento em uma árvore de busca binária cuja chave é um mínimo seguindo ponteiros de filhos da *esquerda* desde a raiz até encontrarmos um valor `NIL`, como mostra a Figura 12.2. O procedimento a seguir, retorna um ponteiro para o elemento mínimo na subárvore enraizada em um nó x dado.

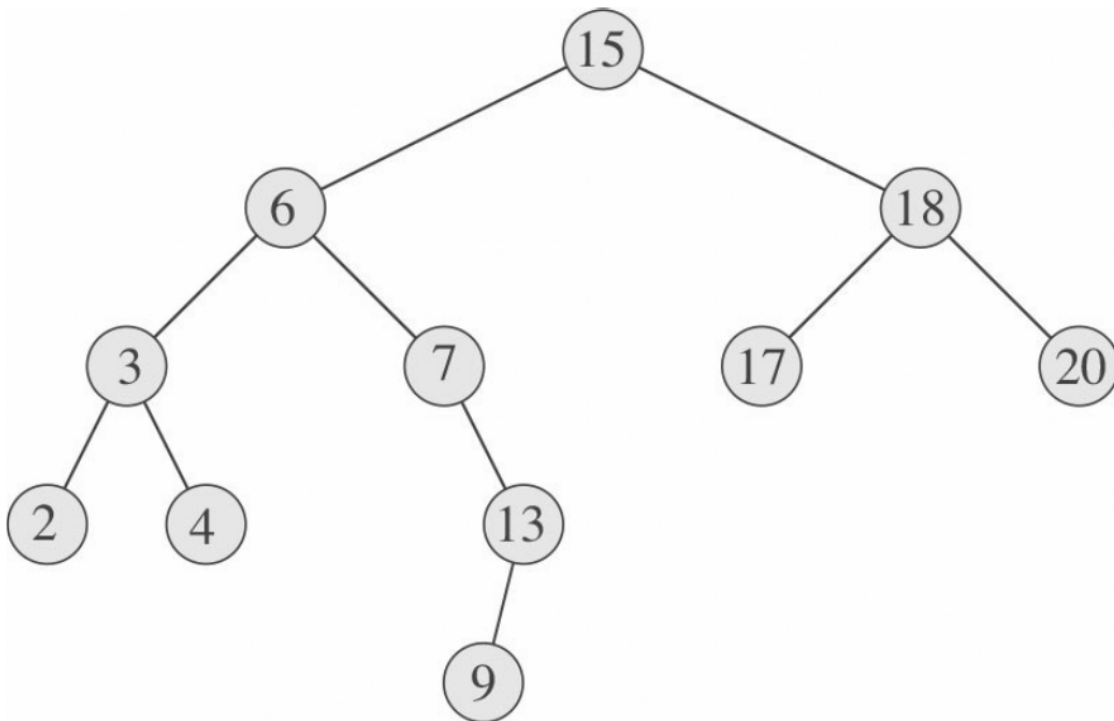


Figura 12.2 Consultas em uma árvore de busca binária. Para procurar a chave 13 na árvore, seguimos o caminho $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$ partindo da raiz. A chave mínima na árvore é 2, que é encontrada seguindo os ponteiros da *esquerda* partindo da raiz. A chave máxima 20 é encontrada seguindo os ponteiros da *direita* partindo da raiz. O sucessor do nó com chave 15 é o nó com chave 17, já que ele é a chave mínima na subárvore direita de 15. O nó com chave 13 não tem nenhuma subárvore direita e, assim, seu sucessor é seu ancestral mais baixo cujo filho à esquerda também é um ancestral. Nesse caso, o nó com chave 15 é seu sucessor.

TREE-MINIMUM(x)

```

1  while  $x.esquerda \neq \text{NIL}$ 
2     $x = x.esquerda$ 
3  return  $x$ 

```

A propriedade de árvore de busca binária garante que TREE-MINIMUM é correto. Se um nó x não tem nenhuma subárvore esquerda, então, visto que toda chave na subárvore direita de x é no mínimo tão grande quanto $x.chave$, a chave mínima na subárvore enraizada em x é $x.chave$. Se o nó x tem uma subárvore esquerda, então, visto que nenhuma chave na subárvore direita é menor que $x.chave$ e toda chave na subárvore esquerda não é maior que $x.chave$, a chave mínima na subárvore enraizada em x pode ser encontrada na subárvore enraizada em $x.esquerda$.

O pseudocódigo para TREE-MAXIMUM é simétrico.

TREE-MAXIMUM(x)

```

1  while  $x.direita \neq \text{NIL}$ 
2     $x = x.direita$ 
3  return  $x$ 

```

Ambos os procedimentos são executados no tempo $O(h)$ em uma árvore de altura h já que, como em TREE-SEARCH, a sequência de nós encontrados forma um caminho simples descendente partindo da raiz.

Sucessor e predecessor

Dado um nó em uma árvore de busca binária, às vezes, precisamos encontrar seu sucessor na sequência ordenada determinada por um percurso de árvore em ordem. Se todas as chaves são distintas, o sucessor de um nó x é o nó com a menor chave maior que $chave[x]$. A estrutura de uma árvore de busca binária nos permite determinar o sucessor de um nó sem sequer comparar chaves. O procedimento a seguir, retorna o sucessor de um nó x em uma árvore de busca binária se ele existir, e NIL se x tem a maior chave na árvore.

TREE-SUCCESSOR(x)

```
1  if  $x.direita \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.direita$ )
3   $y = p[x]$ 
4  while  $y \neq \text{NIL}$  e  $x = y.direita$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

Subdividimos o código para TREE-SUCCESSOR em dois casos. Se a subárvore direita do nó x for não vazia, então o sucessor de x é exatamente o nó da extrema esquerda na subárvore direita de x , que encontramos na linha 2 chamando TREE-MINIMUM($x.direita$). Por exemplo, o sucessor do nó com chave 15 na Figura 12.2 é o nó com chave 17.

Por outro lado, como o Exercício 12.26 pede que você mostre, se a subárvore direita do nó x é vazia e x tem um sucessor y , então y é o ancestral mais baixo de x cujo filho à esquerda é também um ancestral de x . Na Figura 12.2, o sucessor do nó com chave 13 é o nó com chave 15. Para encontrar y , simplesmente subimos a árvore desde x até encontrarmos um nó que seja o filho à esquerda de seu pai; isso é conseguido por meio das linhas 3 a 7 de TREE-SUCCESSOR.

O tempo de execução de TREE-SUCCESSOR em uma árvore de altura h é $O(h)$, já que seguimos um caminho simples para cima na árvore ou, então, um caminho simples para baixo na árvore. O procedimento TREE-PREDECESSOR, que é simétrico de TREE-SUCCESSOR, também é executado no tempo $O(h)$.

Ainda que as chaves não sejam distintas, definimos o sucessor e o predecessor de qualquer nó x como o nó retornado por chamadas feitas a TREE-SUCCESSOR(x) e TREE-PREDECESSOR(x), respectivamente.

Resumindo, demonstramos o teorema a seguir.

Teorema 12.2

Podemos implementar as operações de conjuntos dinâmicos SEARCH, MINIMUM, MAXIMUM, TREE-SUCCESSOR e PREDECESSOR de modo que cada uma seja executada no tempo $O(h)$ em uma árvore de busca binária de altura h .

Exercícios

12.2-1 Suponha que temos números entre 1 e 1.000 em uma árvore de busca binária e queremos procurar o número 363. Qual das seguintes sequências *não* poderia ser a sequência de nós examinados?

- a. 2, 252, 401, 398, 330, 344, 397, 363.
- b. 924, 220, 911, 244, 898, 258, 362, 363.

- c. 925, 202, 911, 240, 912, 245, 363.
- d. 2, 399, 387, 219, 266, 382, 381, 278, 363.
- e. 935, 278, 347, 621, 299, 392, 358, 363.

12.2-2 Escreva versões recursivas de `TREE-MINIMUM` e `TREE-MAXIMUM`

12.2-3 Escreva o procedimento `TREE-PREDECESSOR`.

12.2-4 O professor Bunyan pensa ter descoberto uma notável propriedade de árvores de busca binária. Suponha que a busca da chave k em uma árvore de busca binária termine em uma folha. Considere três conjuntos: A , as chaves à esquerda do caminho de busca; B , as chaves no caminho de busca; e C , as chaves à direita do caminho de busca. O professor Bunyan afirma que quaisquer três chaves $a \in A$, $b \in B$ e $c \in C$ devem satisfazer $a \leq b \leq c$. Dê um contraexemplo menor possível para a afirmação do professor.

12.2-5 Mostre que, se um nó em uma árvore de busca binária tem dois filhos, então seu sucessor não tem nenhum filho à esquerda e seu predecessor não tem nenhum filho à direita.

12.2-6 Considere uma árvore de busca binária T cujas chaves são distintas. Mostre que, se a subárvore direita de um nó x em T é vazia e x tem um sucessor y , então y é o ancestral mais baixo de x cujo filho à esquerda também é um ancestral de x . (Lembre-se de que todo nó é seu próprio ancestral.)

12.2-7 Um método alternativo de executar um percurso de árvore em ordem em uma árvore de busca binária de n nós encontra o elemento mínimo na árvore chamando `TREE-MINIMUM` e depois fazendo $n - 1$ chamadas a `TREE-SUCCESSOR`. Prove que esse algoritmo é executado no tempo $O(n)$.

12.2-8 Prove que, independentemente do nó onde iniciamos em uma árvore de busca binária de altura h , k chamadas sucessivas a `TREE-SUCCESSOR` demoram o tempo $O(k + h)$.

12.2-9 Seja T uma árvore de busca binária cujas chaves são distintas, seja x um nó de folha e seja y seu pai. Mostre que $y.chave$ é a menor chave em T maior que $x.chave$ ou a maior chave em T menor que $x.chave$.

12.3 INSERÇÃO E ELIMINAÇÃO

As operações de inserção e eliminação provocam mudanças no conjunto dinâmico representado por uma árvore de busca binária. A estrutura de dados deve ser modificada para refletir essa mudança, mas de tal modo que a propriedade de árvore de busca binária continue válida. Como veremos, modificar a árvore para inserir um novo elemento é uma operação relativamente direta, mas manipular a eliminação é um pouco mais complicado.

Inserção

Para inserir um novo valor v em uma árvore de busca binária T , utilizamos o procedimento `TREE-INSERT`. O procedimento toma um nó z para o qual $z.chave = v$, $z.esquerda = \text{NIL}$ e $z.direita = \text{NIL}$, e modifica T e alguns dos atributos de z de modo tal que insere z em uma posição adequada na árvore.

TREE-INSERT(T, z)

```
1   $y = \text{NIL}$ 
2   $x = T.\text{raiz}$ 
3  while  $x \neq \text{NIL}$ 
4       $y = x$ 
5      if  $z.\text{chave} < x.\text{chave}$ 
6           $x = x.\text{esquerda}$ 
7      else  $x = x.\text{direita}$ 
8   $z.p = y$ 
9  if  $y = \text{NIL}$ 
10      $T.\text{raiz} = z$            // a árvore  $T$  era vazia
11 else if  $z.\text{chave} < y.\text{chave}$ 
12      $y.\text{esquerda} = z$ 
13 else  $y.\text{direita} = z$ 
```

A Figura 12.3 mostra como TREE-INSERT funciona. Exatamente como os procedimentos TREE-SEARCH e ITERATIVE-TREE-SEARCH, TREE-INSERT começa na raiz da árvore e o ponteiro x e traça um caminho simples descendente procurando um NIL para substituir pelo item de entrada z . O procedimento mantém o *ponteiro acompanhante* y como o pai de x . Após a inicialização, o loop **while** nas linhas 3–7 faz com que esses dois ponteiros se desloquem para baixo na árvore, indo para a esquerda ou a direita, dependendo da comparação de $z.\text{chave}$ com $x.\text{chave}$, até x tornarse NIL. Esse NIL ocupa a posição na qual desejamos colocar o item de entrada z .

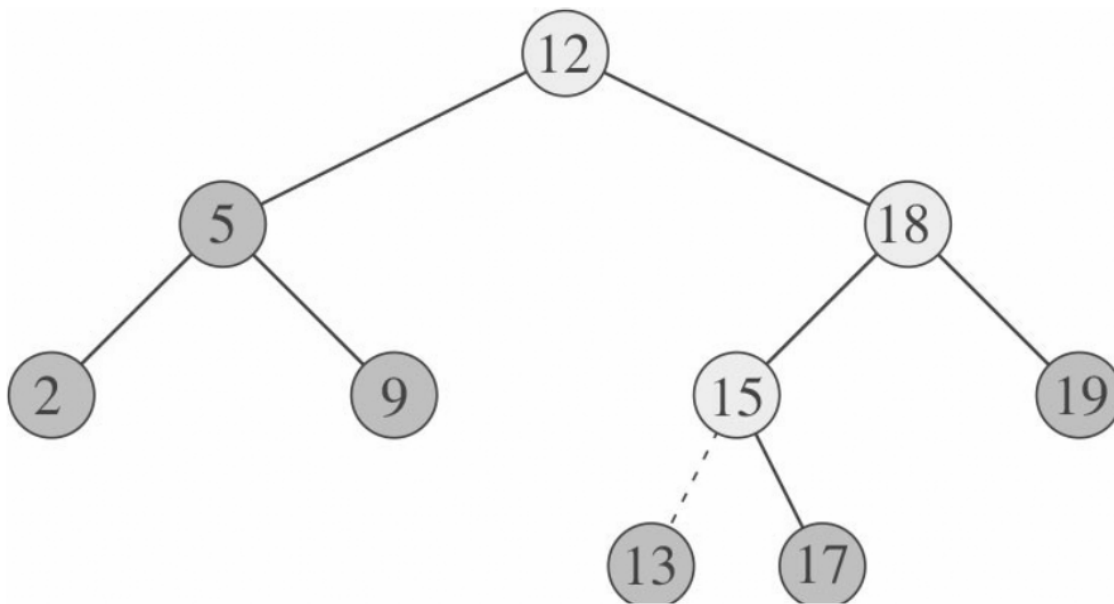


Figura 12.3 Inserção de um item com chave 13 em uma árvore de busca binária. Os nós sombreados em tom mais claro indicam o caminho simples da raiz até a posição em que o item é inserido. A linha tracejada indica a ligação que é acrescentada à árvore para inserir o item.

Precisamos do ponteiro acompanhante y porque, até encontrarmos o NIL ao qual z pertence, a busca desenvolveuse sempre uma etapa à frente do nó que precisa ser mudado. As linhas 8–13 definem os ponteiros que causam a inserção de z .

Do mesmo modo que as outras operações primitivas em árvores de busca, o procedimento `TREE-INSERT` é executado no tempo $O(h)$ em uma árvore de altura h .

Eliminação

A estratégia global para eliminar um nó z de uma árvore de busca binária T tem três casos básicos, mas, como veremos, um deles é um pouco complicado.

- Se z não tem nenhum filho, então simplesmente o removemos modificando seu pai de modo a substituir z por `NIL` como seu filho.
- Se o nó tem apenas um filho, então elevamos esse filho para que ocupe a posição de z na árvore modificando o pai de z de modo a substituir z pelo filho de z .
- Se z tiver dois filhos, encontramos o sucessor de z , y , que deve estar na subárvore direita de z , e obrigamos y a tomar a posição de z na árvore. O resto da subárvore direita original de z tornase a nova subárvore direita de y , e a subárvore esquerda de z tornase a nova subárvore esquerda de y . Esse é o caso complicado porque, como veremos, o fato de y ser ou não o filho à direita de z é importante.

O procedimento para eliminar um dado nó z de uma árvore de busca binária T toma como argumentos ponteiros para T e z . Esse procedimento organiza seus casos de um modo um pouco diferente dos três casos descritos, já que considera os quatro casos mostrados na Figura 12.4.

- Se z não tiver nenhum filho à esquerda (parte (a) da figura), substituímos z por seu filho à direita, que pode ser ou não `NIL`. Quando o filho à direita de z é `NIL`, esse caso trata da situação na qual z não tem nenhum filho. Quando o filho à direita de z é não `NIL`, esse caso manipula a situação na qual z tem somente um filho, que é o seu filho à direita.
- Se z tiver apenas um filho, que é o seu filho à esquerda (parte (b) da Figura), substituímos z por seu filho à esquerda.
- Caso contrário, z tem um filho à esquerda e também um filho à direita. Encontramos o sucessor de z , y , que está na subárvore direita de z e não tem nenhum filho à esquerda (veja o Exercício 12.25). Queremos recortar y de sua localização atual e fazê-lo substituir z na árvore.
 - Se y é o filho à direita de z (parte (c)), substituímos z por y , deixando o filho à direita de y sozinho.
 - Caso contrário, y encontrase dentro da subárvore direita de z , mas não é o filho à direita de z (parte (d)). Nesse caso, primeiro substituímos y por seu próprio filho à direita e depois substituímos z por y .

Para movimentar subárvores dentro da árvore de busca binária, definimos uma subrotina `TRANSPLANT`, que substitui uma subárvore como um filho de seu pai por uma outra subárvore. Quando `TRANSPLANT` substitui a subárvore enraizada no nó u pela subárvore enraizada no nó v , o pai do nó u tornase o pai do nó v , e o pai de u acaba ficando com v como seu filho adequado.

`TRANSPLANT` (T, u, v)

```
1  if  $u.p == \text{NIL}$ 
2       $T.raiz = v$ 
3  elseif  $u == u.p.esquerda$ 
4       $u.p.esquerda = v$ 
5  else  $u.p.direita = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```

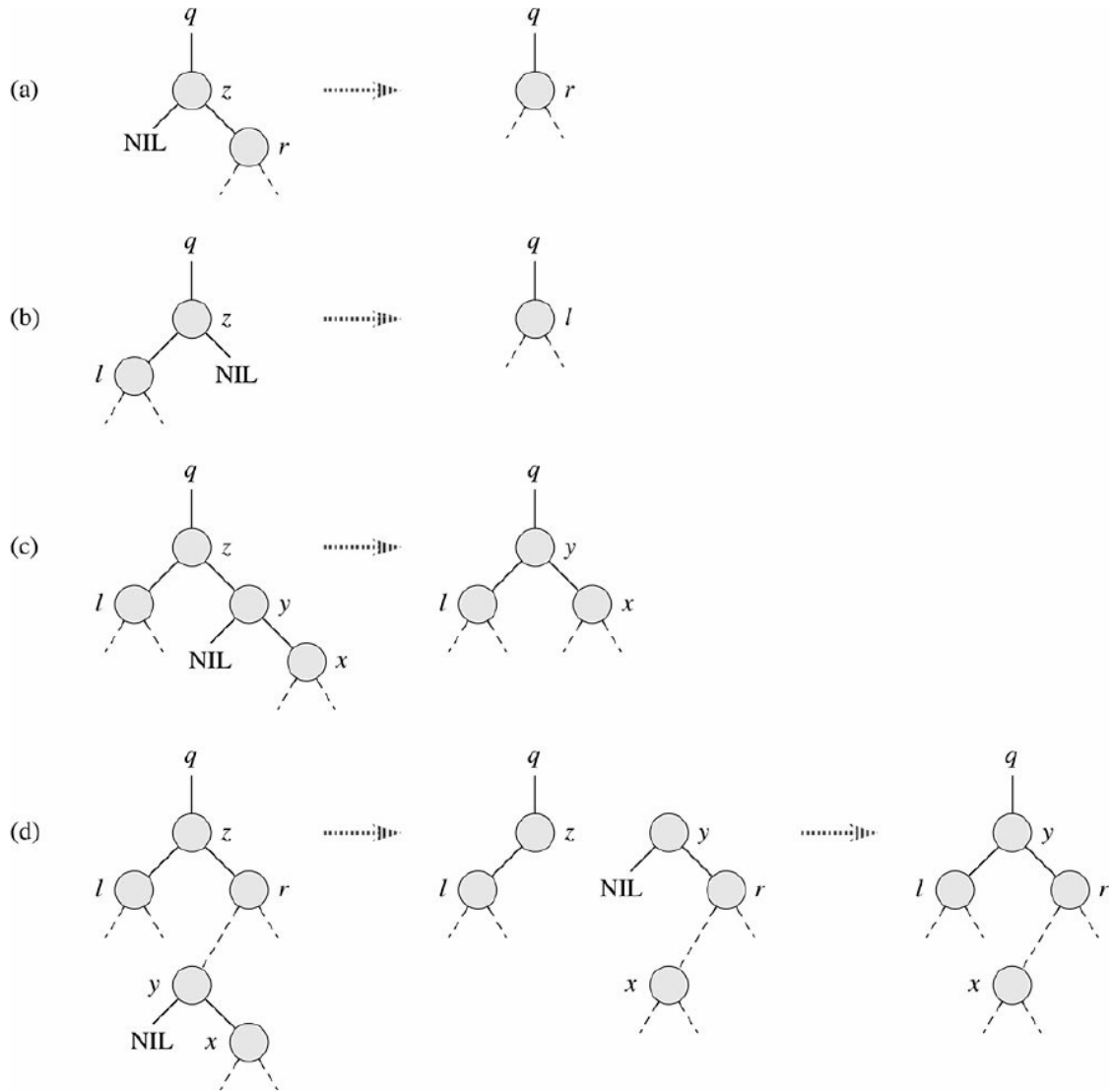


Figura 12.4 Eliminação de um nó de uma árvore de busca binária. O nó z pode ser a raiz, um filho à esquerda do nó q ou um filho à direita de q . **(a)** O nó z não tem nenhum filho à esquerda. Substituímos z por seu filho à direita r , que pode ou não ser NIL. **(b)** O nó z tem um filho à esquerda l mas nenhum filho à direita. Substituímos z por l . **(c)** O nó z tem dois filhos; seu filho à esquerda é o nó l , seu filho à direita é seu sucessor y , e o filho à direita de y é o nó x . Substituímos z por y , atualizando o filho à esquerda de y para que se torne l , mas deixando x como o filho à direita de y . **(d)** O nó z tem dois filhos (o filho à esquerda l e o filho à direita r), e seu sucessor $y \neq r$ encontra-se dentro da subárvore enraizada em r . Substituímos y por seu próprio filho à direita x , e definimos y como pai de r . Então, tomamos y filho de q e pai de l .

As linhas 1–2 tratam do caso no qual u é a raiz de T . Caso contrário, u é um filho à esquerda ou um filho à direita de seu pai. As linhas 3–4 encarregam-se de atualizar $u.p.direita$ se u é um filho à esquerda, e a linha 5 atualiza $u.p.esquerda$ se u é um filho à direita. Permitimos que v seja NIL, e as linhas 6–7 atualizam $v.p$ se v é não NIL. Observe que TRANSPLANT não tenta atualizar $update.v.esquerda$ e $update.v.direita$; fazer ou não fazer isso é responsabilidade do chamador de TRANSPLANT.

Com o procedimento TRANSPLANT em mãos, apresentamos a seguir o procedimento que elimina o nó z da árvore de busca binária T :

TREE-DELETE (T, z)

```
1  if  $z.esquerda == \text{NIL}$ 
2      TRANSPLANT ( $T, z, z.direita$ )
3  elseif  $z.direita == \text{NIL}$ 
4      TRANSPLANT ( $T, z, z.esquerda$ )
5  else  $y = \text{TREE-MINIMUM}(z.direita)$ 
6      if  $y.p \neq z$ 
7          TRANSPLANT ( $T, y, y.direita$ )
8           $y.direita = z.direita$ 
9           $y.direita.p = y$ 
10     TRANSPLANT ( $T, z, y$ )
11      $y.esquerda = z.esquerda$ 
12      $y.esquerda.p = y$ 
```

O procedimento TREE-DELETE executa os quatro casos como descrevemos a seguir. As linhas 1–2 tratam do caso no qual o nó z não tem nenhum filho à esquerda, e as linhas 3–4 tratam do caso no qual z tem um filho à esquerda mas nenhum filho à direita. As linhas 5–12 lidam com os dois casos restantes, nos quais z tem dois filhos. A linha 5 encontra o nó y , que é o sucessor de z . Como z tem uma subárvore direita não vazia, seu sucessor deve ser o nó nessa subárvore que tenha a menor chave; daí a chamada a TREE-MINIMUM($z.direita$). Como observamos antes, y não tem nenhum filho à esquerda. Queremos recortar y de sua localização atual, e ele deve substituir z na árvore. Se y é o filho à direita de z , então as linhas 10–12 substituem z como um filho de seu pai por y e substituí o filho à esquerda de y pelo filho à esquerda de z . Se y não é o filho à direita de z , as linhas 7–9 substituem y como um filho de seu pai pelo filho à direita de y e transforma o filho à direita de z em filho à direita de y , e então as linhas 10–12 substituem z como um filho de seu pai por y e substituem o filho à esquerda de y pelo filho à esquerda de z .

Cada linha de TREE-DELETE, incluindo as chamadas a TRANSPLANT, demora tempo constante, exceto a chamada a TREE-MINIMUM na linha 5. Assim, TREE-DELETE é executado no tempo $O(h)$ em uma árvore de altura h .

Resumindo, provamos o teorema apresentado a seguir.

Teorema 12.3

Podemos implementar as operações de conjuntos dinâmicos INSERT e DELETE de modo que cada uma seja executada no tempo $O(h)$ em uma árvore de busca binária de altura h .

Exercícios

12.3-1 Dê uma versão recursiva do procedimento TREE-INSERT.

12.3-2 Suponha que construímos uma árvore de busca binária inserindo repetidamente valores distintos na árvore. Mostre que o número de nós examinados na busca de um valor na árvore é uma unidade mais o número de nós examinados quando o valor foi inicialmente inserido na árvore.

12.3-3 Podemos ordenar um dado conjunto de n números construindo primeiro uma árvore de busca binária contendo esses números (usando TREE-INSERT repetidamente para inserir os números um a um) e então

imprimindo os números por um percurso de árvore em inordem. Quais são os tempos de execução do pior caso e do melhor caso para esse algoritmo de ordenação?

- 12.3-4** A operação de eliminação é “comutativa” no sentido de que eliminar x e depois y de uma árvore de busca binária resulta na mesma árvore que eliminar y e depois x ? Mostre por que ou dê um contraexemplo.
- 12.3-5** Suponha que, em vez de cada nó x manter o atributo $x.p$, apontando para o pai de x , ele mantém $x.suc$, apontando para o sucessor de x . Dê pseudocódigo para SEARCH, INSERT e DELETE em uma árvore de busca binária T usando essa representação. Esses procedimentos devem funcionar no tempo $O(h)$, onde h é a altura da árvore T . (Sugestão: Seria interessante implementar uma subrotina que retorne o pai de um nó.)
- 12.3-6** Quando o nó z em TREE-DELETE tem dois filhos, podemos escolher o nó y como seu predecessor em vez de seu sucessor. Se fizermos isso, quais outras mudanças serão necessárias em TREE-DELETE? Há quem defenda que uma estratégia justa, que dá igual prioridade ao predecessor e ao sucessor, produz melhor desempenho empírico. Como TREE-DELETE pode ser modificado para implementar tal estratégia justa?

12.4 ★ ÁRVORES DE BUSCA BINÁRIA CONSTRUÍDAS ALEATORIAMENTE

Mostramos que cada uma das operações básicas em uma árvore de busca binária é executada no tempo $O(h)$, onde h é a altura da árvore. Contudo, a altura de uma árvore de busca binária varia à medida que itens são inseridos e eliminados. Se, por exemplo, os n itens são inseridos em ordem estritamente crescente, a árvore será uma cadeia com altura $n - 1$. Por outro lado, o Exercício B.54 mostra que $h \geq \lg n$. Como ocorre com o quicksort, podemos mostrar que o comportamento do caso médio é muito mais próximo do melhor caso que do pior caso.

Infelizmente, sabe-se pouco sobre a altura média de uma árvore de busca binária quando ambas, inserção e eliminação, são utilizadas para criá-la. Quando a árvore é criada somente por inserção, a análise se torna mais tratável. Portanto, vamos definir uma **árvore de busca binária construída aleatoriamente** em n chaves distintas como aquela que surge da inserção das chaves em ordem aleatória em uma árvore inicialmente vazia, onde cada uma das $n!$ permutações das chaves de entrada é igualmente provável. (O Exercício 12.43 pede que você mostre que essa noção é diferente de considerar que toda árvore de busca binária em n chaves é igualmente provável.) Nesta seção, provaremos o teorema apresentado a seguir.

Teorema 12.4

A altura esperada de uma árvore de busca binária construída aleatoriamente em n chaves distintas é $O(\lg n)$.

Prova Começamos definindo três variáveis aleatórias que ajudam a medir a altura de uma árvore de busca binária construída aleatoriamente. Denotamos a altura de uma árvore de busca binária construída aleatoriamente em n chaves por X_n , e definimos a **altura exponencial** $Y_n = 2^{X_n}$. Quando construímos uma árvore de busca binária em n chaves, escolhemos uma chave como a chave da raiz e denotamos por R_n a variável aleatória que contém a **classificação** dessa chave dentro do conjunto de n chaves, isto é, R_n ocupa a posição que essa chave ocuparia se o conjunto de chaves fosse ordenado. O valor de R_n tem igual probabilidade de ser qualquer elemento do conjunto $\{1, 2, \dots, n\}$. Se $R_n = i$, então a subárvore esquerda da raiz é uma árvore de busca binária construída aleatoriamente em $i - 1$ chaves, e a subárvore direita é uma árvore de busca binária construída aleatoriamente em $n - i$ chaves. Como a altura de uma árvore binária é uma unidade maior que a maior das alturas das duas subárvores da raiz, a altura exponencial de uma árvore binária é duas vezes a maior das alturas exponenciais das duas subárvores da raiz. Se sabemos que $R_n = i$, decorre que

$$Y_n = 2 \cdot \max(Y_{i-1}, Y_{n-i}).$$

Como casos-bases temos que $Y_1 = 1$ porque a altura exponencial de uma árvore com 1 nó é $2^0 = 1$ e, por conveniência, definimos $Y_0 = 0$.

Em seguida, definimos variáveis aleatórias indicadoras $Z_{n,1}, Z_{n,2}, \dots, Z_{n,n}$, onde

$$Z_{n,i} = I\{R_n = i\}.$$

Como R_n tem igual probabilidade de ser qualquer elemento de $\{1, 2, \dots, n\}$, decorre que $\Pr\{R_n = i\} = 1/n$ para $i = 1, 2, \dots, n$ e, consequentemente, pelo Lema 5.1, temos

$$E[Z_{n,i}] = 1/n, \quad (12.1)$$

para $i = 1, 2, \dots, n$. Como exatamente um valor de $Z_{n,i}$ é 1 e todos os outros são 0, também temos

$$Y_n = \sum_{i=1}^n Z_{n,i} (2 \cdot \max(Y_{i-1}, Y_{n-i})).$$

Mostraremos que $E[Y_n]$ é um polinômio em n , que, em última análise, implicará $E[X_n] = O(\lg n)$.

Afirmamos que a variável aleatória indicadora $Z_{n,i} = I\{R_n = i\}$ é independente dos valores de Y_{i-1} e Y_{n-i} . Como escolhemos $R_n = i$, a subárvore esquerda (cuja altura exponencial é Y_{i-1}) é construída aleatoriamente nas $i-1$ chaves cujas classificações são menores que i . Essa subárvore é exatamente igual a qualquer outra árvore de busca binária construída aleatoriamente em $i-1$ chaves. Exceto pelo número de chaves que contém, a estrutura dessa subárvore não é afetada de modo algum pela escolha de $R_n = i$; consequentemente, as variáveis aleatórias Y_{i-1} e $Z_{n,i}$ são independentes. Do mesmo modo, a subárvore direita, cuja altura exponencial é Y_{n-i} , é construída aleatoriamente nas $n-i$ chaves cujas classificações são maiores que i . Sua estrutura é independente do valor de $R_n = i$, assim, as variáveis aleatórias Y_{n-i} e $Z_{n,i}$ são independentes. Daí, temos

$$\begin{aligned} E[Y_n] &= E\left[\sum_{i=1}^n Z_{n,i} (2 \cdot \max(Y_{i-1}, Y_{n-i}))\right] \\ &= \sum_{i=1}^n E[Z_{n,i} (2 \cdot \max(Y_{i-1}, Y_{n-i}))] && \text{(por linearidade da esperança)} \\ &= \sum_{i=1}^n E[Z_{n,i}] E[(2 \cdot \max(Y_{i-1}, Y_{n-i}))] && \text{(por independência)} \\ &= \sum_{i=1}^n \frac{1}{n} \cdot E[2 \cdot \max(Y_{i-1}, Y_{n-i})] && \text{(pela equação (12.1))} \\ &= \frac{2}{n} \sum_{i=1}^n E[\max(Y_{i-1}, Y_{n-i})] && \text{(pela equação (C.22))} \\ &\leq \frac{2}{n} \sum_{i=1}^n (E[Y_{i-1}] + E[Y_{n-i}]) && \text{(pelo Exercício C.3-4).} \end{aligned}$$

Visto que cada termo $E[Y_0], E[Y_1], \dots, E[Y_{n-1}]$ aparece duas vezes no último somatório, uma vez como $E[Y_{i-1}]$ e uma vez como $E[Y_{n-i}]$, temos a recorrência

$$E[Y_n] \leq \frac{4}{n} \sum_{i=0}^{n-1} E[Y_i]. \quad (12.2)$$

Usando o método de substituição, agora mostraremos que, para todos os inteiros positivos n , a recorrência (12.2) tem a solução

$$E[Y_n] \leq \frac{1}{4} \binom{n+3}{3}.$$

Ao fazermos isso, usaremos a identidade

$$\sum_{i=0}^{n-1} \binom{i+3}{3} = \binom{n+3}{4}. \quad (12.3)$$

(O Exercício 12.41 pede que você prove essa identidade.)

Para os casos-bases, observamos que os limites $0 = Y_0 = E[Y_0] \leq (1/4) \binom{3}{3} = 1/4$ e $1 = Y_1 = E[Y_1] \leq (1/4) \binom{4}{3} = 1$ são válidos. Para o caso indutivo, temos que

$$\begin{aligned} E[Y_n] &\leq \frac{4}{n} \sum_{i=0}^{n-1} E[Y_i] \\ &\leq \frac{4}{n} \sum_{i=0}^{n-1} \frac{1}{4} \binom{i+3}{3} \quad (\text{pela hipótese de indução}) \\ &= \frac{1}{n} \sum_{i=0}^{n-1} \binom{i+3}{3} \\ &= \frac{1}{n} \binom{n+3}{4} \quad (\text{pela equação 12.3}) \\ &= \frac{1}{n} \cdot \frac{(n+3)!}{4!(n-1)!} \\ &= \frac{1}{4} \cdot \frac{(n+3)!}{3!n!} \\ &= \frac{1}{4} \binom{n+3}{3}. \end{aligned}$$

Limitamos $E[Y_n]$, mas nosso objetivo final é limitar $E[X_n]$. Como o Exercício 12.44 pede que você mostre, a função $f(x) = 2^x$ é convexa (veja página 868). Portanto, podemos empregar a desigualdade de Jensen (C.26), que diz que

$$\begin{aligned} 2^{E[X_n]} &\leq E[2^{X_n}] \\ &= E[Y_n], \end{aligned}$$

da seguinte maneira:

$$\begin{aligned} 2^{E[X_n]} &\leq \frac{1}{4} \binom{n+3}{3} \\ &= \frac{1}{4} \cdot \frac{(n+3)(n+2)(n+1)}{6} \\ &= \frac{n^3 + 6n^2 + 11n + 6}{24}. \end{aligned}$$

Tomando logaritmos de ambos os lados, temos $E[X_n] = O(\lg n)$.

Exercícios

12.4-1 Prove a equação (12.3).

12.4-2 Descreva uma árvore de busca binária em n nós tal que a profundidade média de um nó na árvore seja $\Theta(\lg n)$, mas a altura da árvore seja $(\lg n)$. Dê um limite superior assintótico para a altura de uma árvore de busca binária de n nós na qual a profundidade média de um nó seja $\Theta(\lg n)$.

12.4-3 Mostre que a noção de uma árvore de busca binária escolhida aleatoriamente em n chaves, onde cada árvore de busca binária de n chaves tem igual probabilidade de ser escolhida, é diferente da noção de uma árvore de busca binária construída aleatoriamente dada nesta seção. (*Sugestão:* Faça uma lista de possibilidades quando $n = 3$.)

12.4-4 Mostre que a função $f(x) = 2^x$ é convexa.

12.4-5 ★ Considere uma aplicação de RANDOMIZED-QUICKSORT a uma sequência de n números de entrada distintos. Prove que, para qualquer constante $k > 0$, todas as $n!$ permutações de entrada exceto $O(1/n_k)$ produzem um tempo de execução $O(n \lg n)$.

Problemas

12-1 Árvores de busca binária com chaves iguais

Chaves iguais apresentam um problema para a implementação de árvores de busca binária.

- a. Qual é o desempenho assintótico de TREE-INSERT quando usado para inserir n itens com chaves idênticas em uma árvore de busca binária inicialmente vazia?

Propomos melhorar TREE-INSERT testando antes da linha 5 para determinar se $z.chave = x.chave$, e testando antes da linha 11 para determinar se $z.chave = y.chave$. Se a igualdade for válida, implementamos uma das estratégias a seguir. Para cada estratégia, determine o desempenho assintótico da inserção de n itens com chaves idênticas em uma árvore de busca binária inicialmente vazia. (As estratégias são descritas para a linha 5, na qual comparamos as chaves de z e x . Substitua x por y para chegar às estratégias para a linha 11.)

- b. Mantenha um sinalizador booleano $x.b$ no nó x , e atribua a x o valor $x.esquerdo$ ou $x.direito$, de acordo com o valor de $x.b$, que se alterna entre FALSE e TRUE a cada vez que visitamos x durante a inserção de um nó com a mesma chave que x .
- c. Mantenha em x uma lista de nós com chaves iguais e insira z na lista.
- d. Defina aleatoriamente x como $x.esquerdo$ ou $x.direito$. (Dê o desempenho do pior caso e deduza informalmente o tempo de execução esperado.)

12-2 Árvores digitais

Dadas duas cadeias $a = a_0a_1 \dots a_p$ e $b = b_0b_1 \dots b_q$, onde cada a_i e cada b_j está em algum conjunto ordenado de caracteres, dizemos que a cadeia a é **lexicograficamente menor que** a cadeia b se

1. existe um inteiro j , onde $0 \leq j \leq \min(p, q)$, tal que $a_i = b_i$ para todo $i = 0, 1, \dots, j-1$ e $a_j < b_j$ ou
2. $p < q$ e $a_i = b_i$ para todo $i = 0, 1, \dots, p$.

Por exemplo, se a e b são cadeias de bits, então $10100 < 10110$ pela regra 1 (fazendo $j = 3$) e $10100 < 101000$ pela regra 2. Essa ordenação é semelhante à utilizada em dicionários de idiomas.

A estrutura de dados **árvore digital** mostrada na Figura 12.5 armazena as cadeias de bits 1011, 10, 011, 100 e 0. Quando procuramos uma chave $a = a_0a_1 \dots a_p$, vamos para a esquerda em um nó de profundidade i se $a_i = 0$ e para a direita se $a_i = 1$. Seja S um conjunto de cadeias binárias distintas cujos comprimentos somam n . Mostre como usar uma árvore digital para ordenar S lexicograficamente no tempo $O(n)$. No exemplo da Figura 12.6, a saída da ordenação deve ser a sequência 0, 011, 10, 100, 1011.

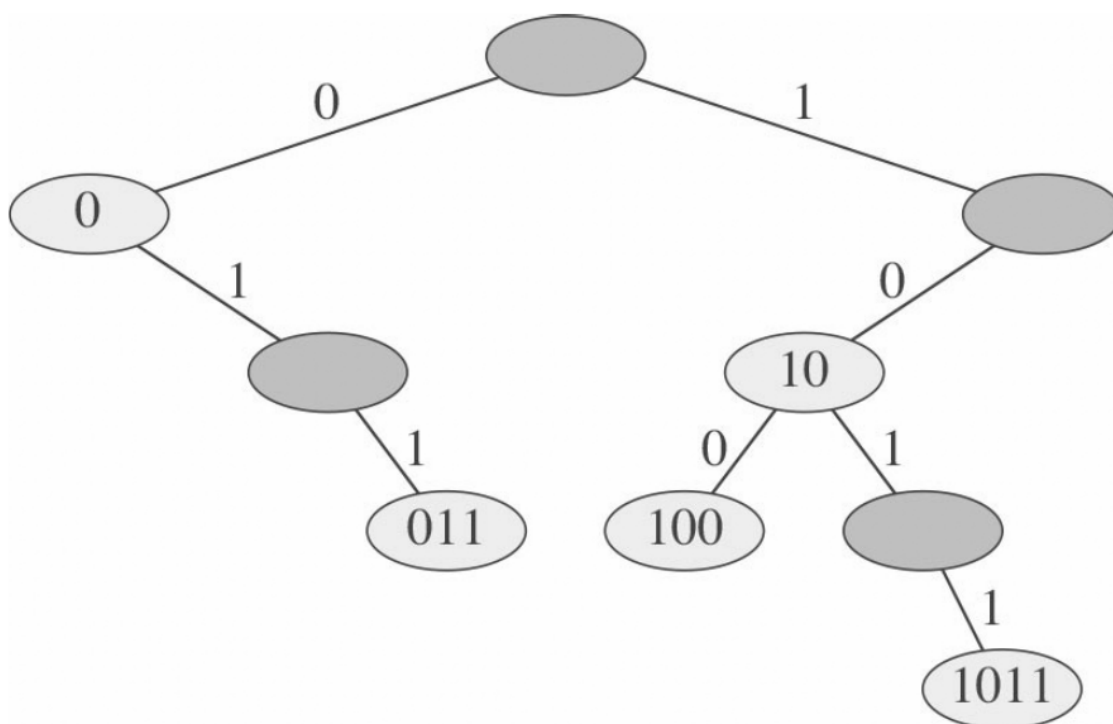


Figura 12.5 Uma árvore digital que armazena as cadeias de bits 1011, 10, 011, 100 e 0. Podemos determinar a chave de cada nó percorrendo o caminho simples da raiz até esse nó. Portanto, não há necessidade de armazenar as chaves nos nós: as chaves são mostradas aqui somente para fins ilustrativos. Os nós estão sombreados em tom mais escuro se as chaves correspondentes a eles não estão na árvore; esses nós estão presentes apenas para estabelecer um caminho até outros nós.

12-3 Profundidade média de nó em uma árvore de busca binária construída aleatoriamente

Neste problema, provamos que a profundidade média de um nó em uma árvore de busca binária construída aleatoriamente com n nós é $O(\lg n)$. Embora esse resultado seja mais fraco que o do Teorema 12.4, a técnica que empregaremos revelará uma semelhança surpreendente entre a construção de uma árvore de busca binária e a execução do algoritmo RANDOMIZED-QUICKSORT da Seção 7.3.

Definimos o **comprimento total de caminho** $P(T)$ de uma árvore binária T como a soma, contando todos os nós x em T , da profundidade do nó x , que denotamos por $d(x, T)$.

a. Demonstre que a profundidade média de um nó em T é

$$\frac{1}{n} \sum_{x \in T} d(x, T) = \frac{1}{n} P(T).$$

Assim, desejamos mostrar que o valor esperado de $P(T)$ é $O(n \lg n)$.

- b.** Denotemos por T_E e T_D as subárvores esquerda e direita da árvore T , respectivamente. Mostre que, se T tem n nós, então

$$P(T) = P(T_E) + P(T_D) + n - 1.$$

- c.** Seja $P(n)$ o comprimento total de caminho médio de uma árvore de busca binária construída aleatoriamente com n nós. Mostre que

$$P(n) = \frac{1}{n} \sum_{i=0}^{n-1} (P(i) + P(n-i-1) + n-1).$$

- d.** Mostre que $P(n)$ pode ser reescrito como

$$P(n) = \frac{2}{n} \sum_{k=1}^{n-1} P(k) + \Theta(n).$$

- e.** Recordando a análise alternativa da versão aleatorizada do quicksort dada no Problema 73, conclua que $P(n) = O(n \lg n)$.

A cada invocação recursiva do quicksort, escolhemos um elemento pivô aleatório para particionar o conjunto de elementos que está sendo ordenado. Cada nó de uma árvore de busca binária particiona o conjunto de elementos que cai na subárvore digital nesse nó.

- f.** Descreva uma implementação do quicksort na qual as comparações para ordenar um conjunto de elementos são exatamente iguais às comparações para inserir os elementos em uma árvore de busca binária. (A ordem em que as comparações são efetuadas pode ser diferente, mas as mesmas comparações devem ser executadas.)

12-4 Número de árvores binárias diferentes

Seja b_n o número de árvores binárias diferentes com n nós. Neste problema, você determinará uma fórmula para b_n , bem como uma estimativa assintótica.

- a.** Mostre que $b_0 = 1$ e que, para $n \geq 1$,

$$b_n = \sum_{k=0}^{n-1} b_k b_{n-1-k}.$$

- b.** Consultando o Problema 44 para a definição de uma função geradora, seja $B(x)$ a função geradora

$$B(x) = \sum_{n=0}^{\infty} b_n x^n.$$

Mostre que $B(x) = xB(x)^2 + 1$ e, consequentemente, um modo de expressar $B(x)$ em forma fechada é

$$B(x) = \frac{1}{2x} (1 - \sqrt{1 - 4x}).$$

A *expansão de Taylor* de $f(x)$ em torno do ponto $x = a$ é dada por

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!} (x-a)^k.$$

onde $f^{(k)}(x)$ é a k ésima derivada de f avaliada em x .

c. Mostre que

$$b_n = \frac{1}{n+1} \binom{2n}{n}$$

(o n ésimo **número de Catalan**) usando a expansão de Taylor de $\sqrt{1-4x}$ em torno de $x = 0$. (Se desejar, em vez de usar a expansão de Taylor, você pode empregar a generalização da expansão binomial (C.4) para expoentes não inteiros n onde, para qualquer número real n e qualquer inteiro k , interpretamos $\frac{(n)}{(k)}$ como $n(n-1) \dots (n-k+1)/k!$ se $k \geq 0$, e 0 em caso contrário.)

d. Mostre que

$$b_n = \frac{4^n}{\sqrt{\pi n^{3/2}}} (1 + O(1/n)).$$

NOTAS DO CAPÍTULO

Knuth [211] contém uma boa discussão de árvores de busca binária simples, bem como de muitas variações. Parece que as árvores de busca binária foram descobertas independentemente por várias pessoas no final da década de 1950. Árvores digitais são frequentemente denominadas “tries”, que vem das letras do meio da palavra *retrieval* em inglês, que significa “recobrar” ou “reaver”. Knuth também as discutiu [211].

Muitos textos, entre eles as duas primeiras edições deste livro, apresentam um método um pouco mais simples de eliminar um nó de uma árvore de busca binária quando ambos os seus filhos estão presentes. Em vez de substituir o nó z por seu sucessor y , eliminamos o nó y mas copiamos sua chave e os dados satélites para o nó z . A desvantagem dessa abordagem é que o nó realmente eliminado poderia não ser o nó passado para o procedimento de eliminação. Se outros componentes de um programa mantiverem ponteiros para nós na árvore, poderiam acabar erroneamente com ponteiros “vencidos” para nós que já foram eliminados. Embora seja um pouco mais complicado, o método de eliminação apresentado nesta edição deste livro garante que uma chamada para eliminar o nó z elimina o nó z e somente o nó z .

A Seção 15.5 mostrará como construir uma árvore de busca binária ótima quando conhecemos as frequências de busca antes da construção da árvore. Isto é, dadas as frequências de busca para cada chave e as frequências de busca para valores que caem entre chaves na árvore, construímos uma árvore de busca binária para a qual um conjunto de buscas que decorre dessas frequências examina o número mínimo de nós.

A prova da Seção 12.4 que limita a altura esperada de uma árvore de busca binária construída aleatoriamente se deve a Aslam [24]. Martínez e Roura [243] dão algoritmos aleatorizados para inserção e eliminação em árvores de busca binária nos quais o resultado de qualquer dessas operações é uma árvore de busca binária aleatória. Contudo, a definição desses autores para uma árvore de busca binária é diferente — apenas ligeiramente — da definição de uma árvore de busca binária construída aleatoriamente dada neste capítulo.